- 128 context length & lower pricing :

Command R features a longer context length, supporting up to 128K tokens in its initial release.

- Strong capabilities across 10 key languages :

The model excels at 10 major languages for global business : English, French, Spanish, Italian, German, Portuguese, Japanese, Korean, Arabic & Chinese.

- Model weights available for research & evaluation!

Cohere For AI is releasing the weights for this version of Command R publicly, so that it can be used for research purposes.

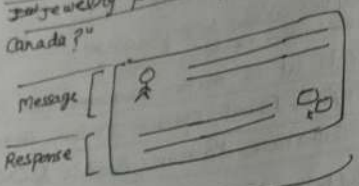
⇒ How an LLM Chatbot Works.

To understand how LLM chatbots work, it's important to develop an understanding of their building blocks. This section focuses on how to build the generative part of a chatbot by looking at how to use a foundational model & added layers of context to generate answers in a conversation style.

↳ The Foundation of an LLM Chatbot

The foundation of an LLM chatbot is an LLM that has been fine-tuned to follow instructions. It can generate a response given a prompt or message from user.

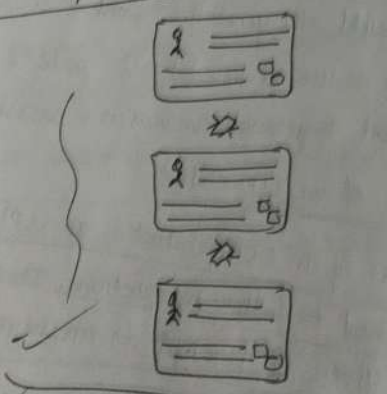
This type of model is tuned to follow instructions and questions, such as "write a headline for my homemade jewelry product" or "what is the capital of Canada?"



A message or prompt returning a model response.

However, the LLM's context is limited to only the last message it receives & it does not consider any previous messages & responses.

Yet, chatbots are characterized by their ability to maintain a conversation with a user, which takes place over multiple interactions.



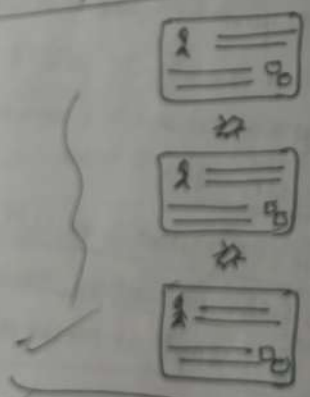
A baseline LLM's context is limited to only the last message it receives.

This type of model is meant to answer interactions and questions, such as "write a headline for my homework" or "what is the capital of Canada?"

A message or prompt returning a model response

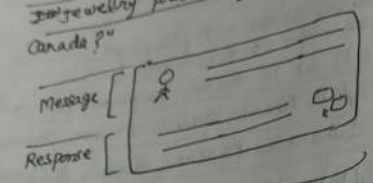
However, the LLM's context is limited to only the last message it receives & it does not consider any previous messages & responses.

Yet, chatbots are characterized by their ability to maintain a conversation with a user, which takes place over multiple interactions.



A baseline LLM's context is limited to only the last message it receives.

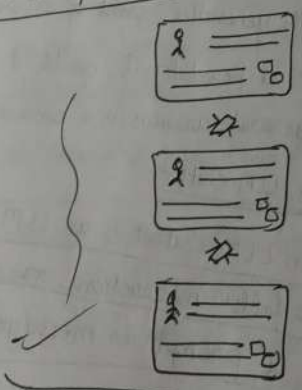
This type of model is tuned to follow instructions and questions, such as "write a headline for my homemade jewelry product" or "what is the capital of Canada?"



A message or prompt returning a model response.

However, the LLM's context is limited to only the last message it receives. & it does not consider any previous messages & responses.

Yet, chatbots are characterized by their ability to maintain a conversation with a user, which takes place over multiple interactions.



A baseline LLM's context is limited to only the last message it receives.

The goal of a chatbot
a sequence of interactio
the chatbot to hold an
the model's response c
interactions instead
every time.

⇒ How to Build

working off of the
can layer together m
& create a memo
First, we add a
A preamble conta
chatbot's' respons
a persona, sty
chatbot to adop
used to enco
& professional
something like
assist human
& professiona

Then, we appen
which becomes
we append th
message to

The goal of a chatbot is to solve this prob^m by linking a sequence of interactions into a single instance, allowing the chatbot to hold an outgoing conversation. In doing so, the model's response can keep a memory of all the previous interactions instead of having to start from scratch every time.

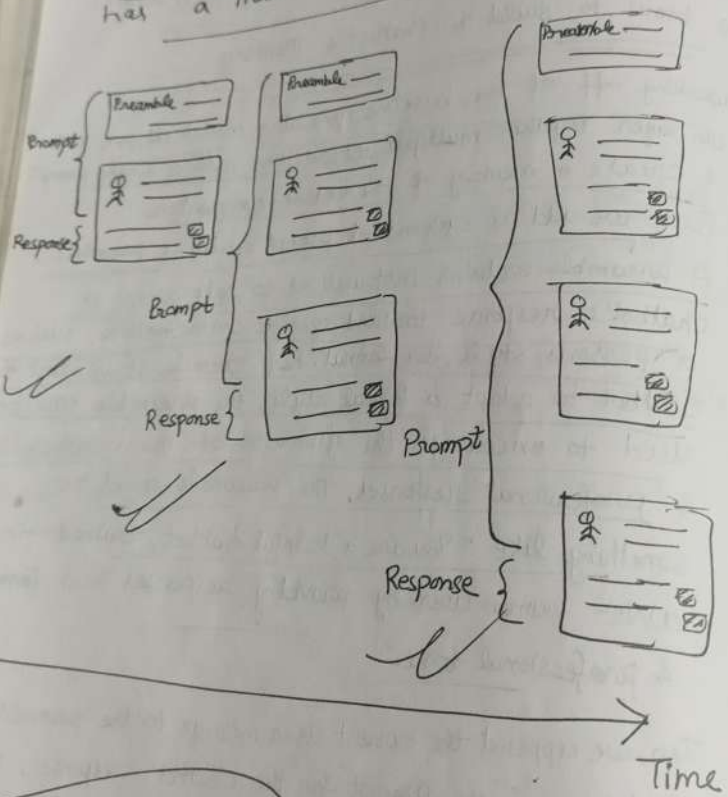
⇒ How to Build a Chatbot's Memory.

Working off of the baseline generation model above we can layer together multiple interactions into a single prompt & create a memory of the entire conversation.

First, we add a system-level prompt called a preamble. A preamble contains instructions to help steer a chatbot's response toward specific characteristic, such as a persona, style, or format. For example, if we want the chatbot to adopt a formal style, the preamble can be used to encourage the generation of more business-like & professional responses. The preamble could be something like "You are a helpful chatbot, trained to assist human users by providing responses in a formal & professional tone.".

Then, we append the current user message to the preamble, which becomes the prompt for the chatbot's response. Next, we append the chatbot response & the following user message to the prompt.

We can repeat this step for any no. of interactions until we reach the model's maximum context length. Context length is the total no. of tokens taken up by the prompt & response, and each model has a maxm context length that it can support.



Building a conversation by stitching multiple prompt-response pairs together.

This multi-turn fra
to hold the
to finish.

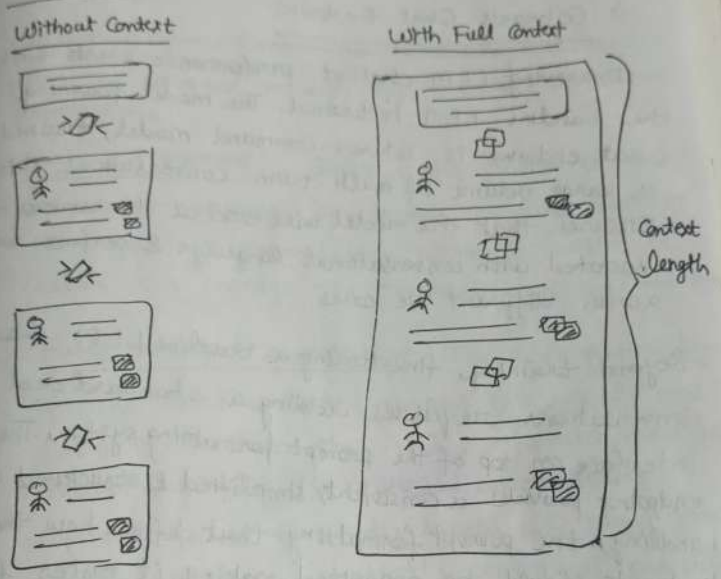
Without Context

Multi-tur

↳ However, bu
sufficient.

Chatbots need
To create a rob
& reliable o/p
to conversati
fine-tuning

This multi-turn framework is what gives chatbots the ability to hold the full context of conversation from start to finish.



Multi-turn conversations can happen when full context is available

→ However, building on top of a baseline LLM alone is not sufficient.

Chatbots need to perform well in a wide range of scenarios. To create a robust chatbot that consistently generates high-quality & reliable o/p, the baseline LLM needs to be adapted specifically to conversations. This means taking the baseline model & fine-tuning it further with a large volume of conversational data.

This is what forms the foundation of Cohere's Chat endpoint.

⇒ Cohere's Chat Endpoint

Improving LLM chatbot performance starts with how the baseline LLM is trained. The model powering the Chat endpoint is Cohere's Command model, trained with a large volume of multi-turn conversations. This ensures that the model will excel at the various nuances associated with conversational language & perform well across different use cases.

Beyond training, fine-tuning a baseline LLM for conversations requires adding a standardized interface on top of the prompt formatting system. The chat endpoint provides a consistent, simplified & structured way of handling the prompt formatting that defines how the prompt ~~to~~ Inputs should be organized, making it easier for developers to build chatbot applications. This added layer includes a fixed abstraction & schema, providing more stability to scale & build app's on top of the foundation model.

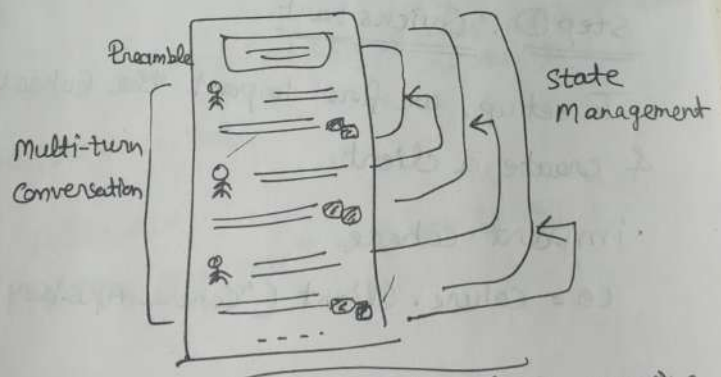
↳ The chat endpoint includes all the elements required for an LLM chatbot (as discussed in the previous sections), exposing a simple interface for developers. It consists of the following key components:

- Preamble managemen
endpoint's default
own preambles.
- multi-turn conve
upon the comm
conversations.
- State manag
conversation
the endpo
feature co
- Fully-m
of the
item
Everyth
At th
contre

Mu
Co

The chat endpo
a simple int

- Preamble management: Developers can opt to use the endpoint's default preamble or override it with their own preambles.
- Multi-turn conversations: The chat endpoint builds upon the command model by enabling multi-turn conversations.
- State management: State mgmt preserves the conversation memory. Developers can either leverage the endpoint's conversation history persistence feature or manage the conversation history themselves
- Fully-managed conversation: The abstraction layer of the chat endpoint means there's only one item to send to the API: the user message. Everything else is managed automatically. At the same time, developers who want greater control over a chatbot's configurat^n can still do so.



The chat endpoint takes care of the underlying logic, exposing a simple interface for developers.

↳ Cohere's Command model & chat endpoint offer powerful tools for developers looking to harness the potential of LLMs in their app^ns. Command's capabilities, coupled with the chat endpoint's developer-friendly interface, allow developers to build text generat^n app^ns across diverse business app^ns.

Building a Chatbot

Here, you'll learn how to build a simple chatbot from scratch using the chat endpoint & you'll explore features like defining preambles, streaming & state mgmt.

↳ Here, we build a simple chatbot that can respond to user messages & maintain the context of the conversation.

↳ Step-by-Step Guide

Step ①: Quickstart

To setup, we first import the Cohere module & create a client.

```
import cohere
co = cohere.Client("Cohere_API_key")
```